

HW5

Part 1: Anime Face Generation

- 1-1 Denoising Process

- init()

```
if beta_schedule == 'linear':
    beta_schedule_fn = linear_beta_schedule
elif beta_schedule == 'cosine':
    beta_schedule_fn = cosine_beta_schedule
else:
    raise ValueError(f'unknown beta schedule {beta_schedule}')

betas = beta_schedule_fn(timesteps, **schedule_fn_kwargs) #  $\beta_t$ 
alphas = 1. - betas #  $\alpha_t = 1 - \beta_t$ 
alphas_bar = torch.cumprod(alphas, dim=0) #  $\bar{\alpha}_t = \prod \alpha_i$ 

return betas, alphas, alphas_bar
```

- add_noise_forward()

The process of adding T-times Gaussian distribution to transform the image into pure Gaussian noise.

```
#####
# TODO1-1-2: Denoising Process - Forward function
# Begin your code

sqrt_alphas_bar = torch.sqrt(extract(self.alphas_bar, t, x_start.shape))
sqrt_one_minus_alphas_bar = torch.sqrt(1. - extract(self.alphas_bar, t, x_start.shape))
return sqrt_alphas_bar * x_start + sqrt_one_minus_alphas_bar * noise

# End your code
#####
```

- denoise_backward()

By predicting the inverse distribution of the added noise in the forward process, the step of transforming pure Gaussian noise into an image is achieved.

```
alphas_bar = self.alphas_bar
alphas_bar_prev = self.alphas_bar_prev

t = torch.full((pred_x_start.shape[0],), t, device=pred_x_start.device, dtype=torch.long)
t_prev = torch.full((pred_x_start.shape[0],), t_prev, device=pred_x_start.device, dtype=torch.long)

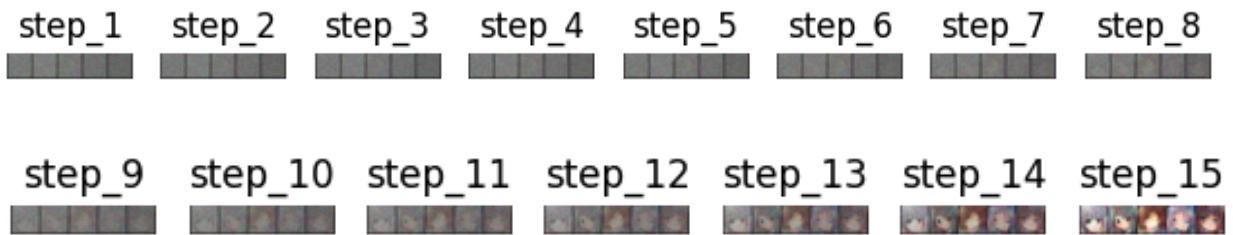
# extract and reshape for broadcasting
a_t = extract(alphas_bar, t, pred_x_start.shape)
a_prev = extract(alphas_bar_prev, t, pred_x_start.shape)

sigma = eta * torch.sqrt((1 - a_prev) / (1 - a_t)) * torch.sqrt(1 - (a_t / a_prev))

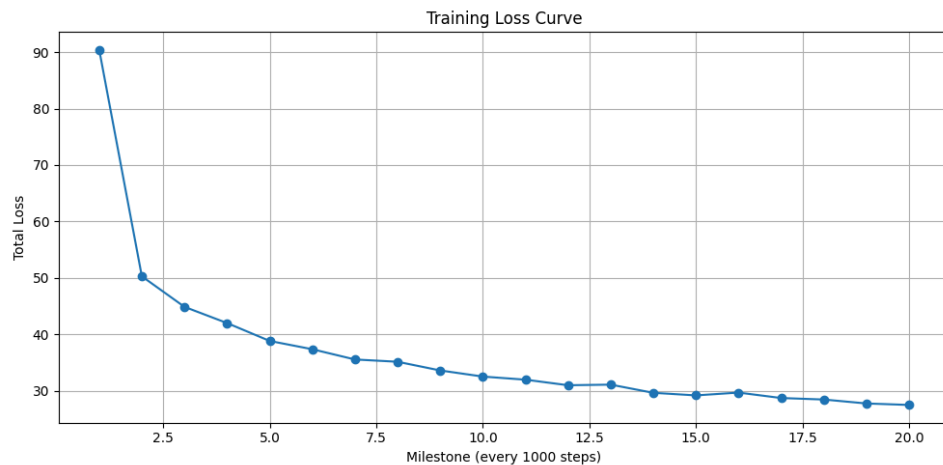
#  $\mu_t(x_t)$ 
mean = torch.sqrt(a_prev) * pred_x_start + \
    torch.sqrt(1 - a_prev - sigma**2) * pred_noise

return mean + sigma * noise
```

- 1-2 Result Visualization
 - Denoising progress



- Loss curve



By the above result, we successfully built the process of generating the image.

- Result & Discussion
 - When using **small sampling steps** (e.g., 10–20), the generated faces were often **over-smoothed**, lacking fine textures (e.g., hair detail, eye sharpness).
 - With default channels = 32, **checkerboard artifacts** and **blurry facial features** appeared, especially inside profiles.

Final hyperparameter :

```
##### Hyper-Parameters #####
import torchvision.utils as vutils
seed = 114514
path = '/kaggle/input/diffusion/faces/faces'
IMG_SIZE = 64
batch_size = 64
train_num_steps = 20000
lr = 1e-4
grad_steps = 1
ema_decay = 0.999

channels = 64
dim_mults = (1, 2, 4, 8)

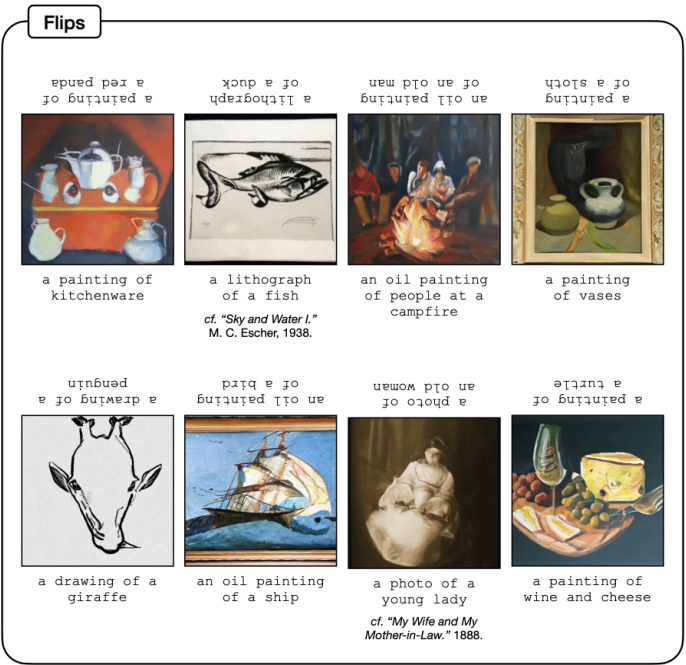
timesteps = 1000
sampling_timesteps = 20
beta_schedule = 'cosine'
```

My strategy include:

- Increasing model depth (via channels and dim_mults) was critical in reducing artifacts and over-smoothing.
- Cosine schedule and EMA significantly improved stability.

Part 2: Optical illusion Generation

In this part, we are asked to do the work below.



2-1 Prompt Design

```
prompt_1 = 'painting of a crow'
prompt_2 = 'painting of a black cat'
```

2-2 Viewing Transformation

```
class IdentityView:
```

```

def __init__(self):
    pass

def view(self, im):
    return im

def inverse_view(self, noise):
    return noise

class Rotate180View:
    def __init__(self):
        pass

    def view(self, im):
        return TF.hflip(TF.vflip(im))

    def inverse_view(self, noise):
        return TF.hflip(TF.vflip(noise))

views = [IdentityView(), Rotate180View()]

```

- 2-3 Denoising Operation

This function is a two-stage common denoising core implementation, mainly including the following steps:

1. Input preparation and verification

- Verify that prompt_embeds is a positive and negative prompt (CFG) with a total of $2 \times$ the number of viewpoints
- If it is Stage 2, preprocess noise_level and upscaled image

2. Denoising time loop

For each diffusion timestep t , execute in sequence:

- Use view.view(x) to convert the noise image to an observed image under a certain viewpoint
- If it is Stage 2, concat with upscaled image
- Copy a copy as uncond conditional input (CFG)
- Call model.unet based on prompt_embeds and (optional) noise_level
- Output is [uncond_pred, cond_pred], which may include variance
- Disassemble noise and predicted_variance

- Use `view.inverse_view()` to restore the guided noise to the original view
- Average the noise prediction results under all views and concatenate with variance
- Call `scheduler.step()` to perform a reverse diffusion on x_t to get x_{t-1}
- 2-4 Evaluation Baseline
Result:

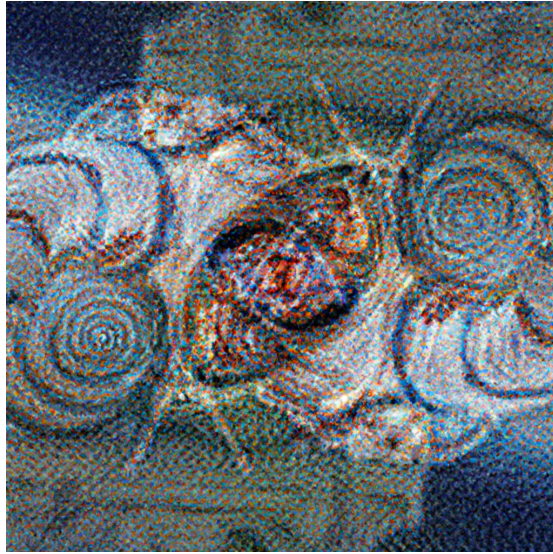


```
Prompt: painting of a crow
CLIP Score: 0.3446

Prompt: painting of a black cat
CLIP Score: 0.3430
```

- Result & Discussion
 - Since I believe two prompts must be similar to get better performance, I pick crow and blackcat. In order to make it in the same style. I constraint a painting style.

- fail case:



Part 3: Link

https://drive.google.com/drive/folders/15Ed2vEWI__SxakUKrOuB4YWGgRMzSX0o?usp=sharing